

PART A THEORY

1. Explain with example what a greedy choice is.

A greedy choice refers to an approach in problem-solving where you make the locally optimal choice at each step with the hope that these local choices lead to a globally optimal solution.

Example: Coin Change Problem

Imagine you have a set of coin denominations, such as {1, 5, 10, 25}, and you need to make a total of 30. A greedy choice would be to pick the largest coin possible at each step:

1. First, choose the largest coin less than or equal to 30, which is 25.
2. Next, choose the largest coin less than or equal to the remaining amount ($30 - 25 = 5$), which is 5.

The greedy approach leads to the solution of using two coins: 25 and 5. It works optimally for this problem, as this combination minimizes the number of coins.

However, greedy choices don't always lead to an optimal solution for every problem. In some cases, a more complex strategy (such as dynamic programming or backtracking) is necessary. But for problems where a greedy choice works, it's a fast and efficient method.

2. Prove that activity selection problem has greedy choice property.

The **Activity Selection Problem** has the **Greedy Choice Property** because selecting the activity that finishes first leads to an optimal solution.

Proof:

1. **Greedy Choice:** In the Activity Selection Problem, the greedy choice is to select the activity that finishes the earliest, ensuring it doesn't overlap with previously selected activities.
2. **Why It's Greedy:** By picking the earliest finishing activity, we maximize the remaining time for selecting future activities. This decision is made without looking ahead, which is the essence of a greedy algorithm.
3. **Optimality of Greedy Choice:**
 - Let's assume there is an **optimal solution** OO with more activities than the greedy solution GG. If this were true, we could swap one of the activities from OO with the first activity from GG, since the greedy choice finishes earlier and doesn't overlap with the remaining activities.
 - This swap does not reduce the number of activities selected. Therefore, by making the greedy choice, we are still able to achieve an optimal solution.
4. **Conclusion:** The greedy choice (selecting the activity that finishes first) ensures the maximum possible number of non-overlapping activities are selected, proving that the Activity Selection Problem has the **Greedy Choice Property**.

3. Greedy strategy does not give the optimal solution for 0/1 knapsack. Do you agree with this statement? Justify your answer with example.

Yes, I agree with the statement that the **Greedy strategy does not give the optimal solution for the 0/1 Knapsack Problem**. Here's why:

0/1 Knapsack Problem:

In the 0/1 Knapsack Problem, we are given a set of items, each with a weight and a value, and a knapsack with a capacity limit. The goal is to determine the maximum value we can fit in the knapsack by either including or excluding each item (i.e., 0 or 1 decision).

Greedy Strategy:

A common greedy strategy for the knapsack problem is to choose items based on the **value-to-weight ratio** (value/weight), starting with the item that has the highest ratio. However, this doesn't always yield the optimal solution for the 0/1 Knapsack Problem.

Why It Doesn't Work:

The greedy approach doesn't always work because it doesn't consider the combinations of items that might result in a better overall solution. The knapsack has a fixed capacity, and selecting the item with the best value-to-weight ratio can sometimes lead to a suboptimal solution if other combinations of items offer a higher total value.

Example:

Items:

Item	Weight	Value	Value/Weight Ratio
------	--------	-------	--------------------

1	2	3	1.5
2	3	4	1.33
3	4	5	1.25
4	5	7	1.4

Knapsack Capacity: 5

Greedy Approach (Value/Weight Ratio):

1. **Item 1** has the highest ratio (1.5). It fits, so we select it.
2. **Item 4** has the next highest ratio (1.4), but it doesn't fit anymore (total weight exceeds 5).

The greedy algorithm would stop with just **Item 1** (weight 2, value 3).

Optimal Solution:

Instead of just taking the item with the highest ratio, the optimal solution is to select **Item 2** (weight 3, value 4) and **Item 1** (weight 2, value 3). This combination gives a total value of **7** and a total weight of **5**, which perfectly fills the knapsack.

4. Explain the difference between Dynamic Programming and Greedy Strategy with examples.

Greedy Strategy:

1. **Approach:**
 - Makes the **best choice** at each step, hoping it leads to the best overall solution.
 - Once a decision is made, it is never revisited.
2. **Optimality:**
 - Works **only for certain problems** where local choices lead to the best global solution.
 - Not always optimal for all problems.
3. **Example:**
 - **Fractional Knapsack:** You pick items with the highest value-to-weight ratio. It works well here because you can take fractions of items.
4. **Time Complexity:**
 - Usually faster (often linear or logarithmic time).

Dynamic Programming (DP):

1. **Approach:**

- Breaks the problem into **smaller subproblems**, solves them, and combines their results.
- **Revisits decisions** and stores solutions to avoid redundant work.

2. **Optimality:**

- Always gives the **optimal solution** if the problem has overlapping subproblems and optimal substructure.

3. **Example:**

- **0/1 Knapsack:** You decide whether to include each item or not, considering all combinations to find the best solution.

4. **Time Complexity:**

- More time-consuming than greedy (polynomial time, often).

Key Differences:

Feature	Greedy	Dynamic Programming
Approach	Makes a quick choice at each step.	Solves all subproblems and combines results.
Optimality	Not always optimal.	Always gives the optimal solution.
Example	Fractional Knapsack.	0/1 Knapsack, Fibonacci.
Time Complexity	Faster (linear/logarithmic).	Slower (polynomial).

PART B THEORY

1. In Euclid's algorithm, what happens when the remainder of the division becomes 0?

In **Euclid's Algorithm** for finding the **Greatest Common Divisor (GCD)** of two numbers, the algorithm works by repeatedly dividing the larger number by the smaller number and replacing the larger number with the remainder. This process continues until the remainder becomes 0.

What happens when the remainder becomes 0?

- When the remainder becomes **0**, the **last non-zero remainder** is the **GCD** of the two numbers.
- This is because the GCD is the largest number that divides both original numbers without leaving a remainder. Once the remainder reaches 0, the divisor at that point is the GCD.

Example:

Let's calculate the GCD of 48 and 18 using Euclid's algorithm:

1. **Step 1:** Divide 48 by 18:
 - $48 \div 18 = 2$ remainder 12
 - New pair: (18, 12)
2. **Step 2:** Divide 18 by 12:
 - $18 \div 12 = 1$ remainder 6
 - New pair: (12, 6)
3. **Step 3:** Divide 12 by 6:
 - $12 \div 6 = 2$ remainder 0
 - Remainder is 0, so the process stops here.

At this point, the **GCD** is **6** because the last non-zero remainder before the remainder became 0 was 6.

2. What is the difference between the Floyd-Warshall algorithm and Dijkstra's algorithm for finding the shortest path between two vertices in a weighted graph?

Great question! Both **Floyd-Warshall** and **Dijkstra's** algorithms are used to find the shortest paths in a **weighted graph**, but they differ significantly in their approach, use case, and performance. Here's a detailed comparison:

Floyd-Warshall vs Dijkstra's Algorithm

Feature	Floyd-Warshall	Dijkstra's Algorithm
Purpose	Finds shortest paths between all pairs of vertices	Finds shortest path from a single source to all others

Feature	Floyd-Warshall	Dijkstra's Algorithm
Graph Type	Works on directed/undirected, weighted graphs	Works on directed/undirected, weighted graphs
Negative Weights	Yes (but no negative cycles)	No (fails with negative edge weights)
Time Complexity	$O(V^3)$	$O((V + E) \log V)$ with a min-priority queue (heap)
Space Complexity	$O(V^2)$ (uses a 2D matrix)	$O(V)$ (distance array), $O(E)$ for adjacency list
Approach Type	Dynamic Programming	Greedy Algorithm
Best Use Case	When you need all-pairs shortest paths	When you need single-source shortest path
Implementation Simplicity	Simple matrix-based loops	Requires priority queue or min-heap for efficiency

Example Use Cases

- **Floyd-Warshall:**
 - Route planning between **every pair of cities**
 - Analyzing **transitive closure** in graphs
 - Detecting **negative cycles**
- **Dijkstra:**
 - Finding shortest path from a starting city to all others (e.g., **Google Maps** routing from a specific location)
 - Network packet routing from one node

3. Clearly define the following terms.

(i) Polynomial time algorithm (ii) Decision problem (iii) Optimization problem (iv) Intractable problem (v) Tractable problem (vi) Deterministic machine

(i) Polynomial Time Algorithm

A **polynomial time algorithm** is an algorithm whose **running time** is **bounded by a polynomial function** of the size of the input (e.g., $O(n)$, $O(n^2)$, $O(n^3)$, etc.).

These algorithms are considered **efficient** and **practically solvable**.

Example: Bubble Sort runs in $O(n^2)$ time.

(ii) Decision Problem

A **decision problem** is a problem that has a **yes/no** answer. It asks whether a given input satisfies certain conditions.

Example:

"Does this graph contain a cycle?" → Yes or No.

(iii) Optimization Problem

An **optimization problem** requires finding the **best solution** (maximum or minimum) from a set of feasible solutions, according to some objective function.

Example:

"Find the shortest path from A to B in a graph."

(iv) Intractable Problem

An **intractable problem** is a problem that **cannot be solved in polynomial time**.

It usually requires **exponential or super-polynomial time**, making it **impractical** to solve as the input size grows.

Example:

Traveling Salesman Problem (exact solution) — time complexity increases rapidly.

(v) Tractable Problem

A **tractable problem** is one that **can be solved in polynomial time**.

These are considered **efficiently solvable** and manageable even for large inputs.

Example:

Dijkstra's algorithm for shortest path ($O(V + E \log V)$).

(vi) Deterministic Machine

A **deterministic machine** is a theoretical model of computation where the **next state is fully determined** by the current state and input.

There is **no randomness** or guesswork — the same input always leads to the same output.

Example:

Deterministic Turing Machine (DTM), used in defining class **P**.

4. Explain why NP-complete class is important in complexity theory.

Importance of NP-Complete Class in Complexity Theory

The **NP-Complete class** plays a **central role** in complexity theory because it helps us understand the **boundary between easy and hard problems** in computer science.

Reasons Why NP-Complete Class Is Important:

1. Defines the Hardest Problems in NP

- NP-Complete problems are the **most difficult problems** in the NP class.
- If **any one** NP-Complete problem can be solved in polynomial time, **all** NP problems can be solved efficiently.
- This means:

$$P = NP$$

2. Helps Explore the P vs NP Question

- The P vs NP question is one of the **biggest open problems** in computer science.
- NP-Complete problems are at the **core of this question**.

3. Useful in Problem Classification

- It helps computer scientists classify problems into **tractable (easy)** and **intractable (hard)**.
- If a new problem is shown to be NP-Complete, we know it is **unlikely to have an efficient algorithm**.

4. Focus for Research

- Many researchers focus on finding **approximation, heuristics, or special cases** for NP-Complete problems.
- These problems often appear in **real-world fields** like:
 - Scheduling
 - Network design
 - Logistics
 - Cryptography
 - AI and Machine Learning

5. Polynomial-Time Reductions

- The concept of **reducing one problem to another** in polynomial time was developed through NP-Complete theory.
- It allows us to relate the difficulty of problems.